

dj



Templates et Vues

Les Templates DJANGO

- **Les Variables complexes**
 - L'opérateur « . » a trois significations dans le langage de templates :
 - Consultation de dictionnaire
 - Consultation d'attribut ou de méthode (les méthodes n'ont pas de paramètres)
 - Consultation d'indice numérique

```
# dans app/templates/app/index.html
```

```
this project is {{ project.name }}. this app is {{ project.app }}.
```

```
# dans app/views.py
```

```
from django.template.response import TemplateResponse
```

```
def index(request):  
    return TemplateResponse(request, 'app/index.html', {"project": {"name": "project",  
        "app": "app"}})
```

Les Templates DJANGO

- Auto echappement des variables
 - Les variables de templates sont automatiquement échappées pour éviter les injections de code javascript ou de balise (faille XSS)
 - On peut désactiver cet échappement de plusieurs façons
 - Sur une variable : `{{ variable|safe }}`
 - Sur un bloc :

```
{% autoescape off %}  
Hello {{ name }}  
{% endautoescape %}
```

Les Templates DJANGO

- Les Filtres

- Les filtres, de format `{{ variable|filtre }}`, opèrent un traitement sur la variable associée. Ce sont des « helpers » de templates
- Les filtres peuvent s'enchaîner : `{{ variable|filtre1|filtre2 }}`
- Les filtres peuvent posséder des paramètres : `{{ variable|filtre:param }}`
- Le développeur peut créer ses propres filtres

dans `app/templates/app/index.html`

this project is `{{ project.name|lower }}`. this app is `{{ project.app }}`.
next content : `{{ content|default: "nothing" }}`

<https://docs.djangoproject.com/fr/3.1/ref/templates/builtins/#ref-templates-builtins-filters>

Les Templates DJANGO

- Les Balises

- Les balises, de format `{% tag %}` assument plusieurs rôles distinct :
- Structures de contrôle : instructions conditionnelles « if » et boucles d'itération « for »
- Ajouts / Remplacement de contenus : « block » et « extends »
- Extension du langage de balise : « load »

```
{% extends "base_generic.html" %}
{% block content %}
<h1>{{ section.title }}</h1>

{% for story in story_list %}
<h2>
  <a href="{{ story.get_absolute_url }}">
    {{ story.headline|upper }}
  </a>
</h2>
<p>{{ story.tease|truncatewords:"100" }}</p>
{% endfor %}
{% endblock %}
```

<https://docs.djangoproject.com/fr/3.1/ref/templates/builtins/>

Les Templates DJANGO

- Les balises if, elif et else

Évalue une variable, et si cette variable vaut True, le contenu du bloc est affiché :

```
{% if article_list %}

    Number of articles: {{ article_list|length }}

{% elif comment_list %}

    {{ comment_list[0]|truncatewords:30 }}

{% else %}

    No contents.

{% endif %}
```


Les Templates DJANGO

- La balise for

Boucle sur chaque élément d'une liste.

```
<ul>
```

```
{% for article in article_list %}
```

```
<li>{{ article.name }}</li>
```

```
{% endfor %}
```

```
</ul>
```

Les Templates DJANGO

- L'héritage de templates : block
- On peut définir des templates de « layout », possédant des balises « block » qui définissent des zones de contenus, templates destinés à être étendus par des templates enfants, qui vont surcharger le contenu des blocks

```
# layout.html
<!DOCTYPE html>
<html lang="fr">
<head>
  <title>{% block title %} titre générique {% endblock %}</title>
</head>
<body>
  <div id="content">
    {% block content %}{% endblock %}
  </div>
</body>
</html>
```


Les Templates DJANGO

- L'héritage de templates : extends

```
# child.html
{% extends "layout.html" %}

{% block title %} Mon blog {% endblock %}

{% block content %}

    {% for article in article_list %}

        <h2>{{ article.title }}</h2>

        <p>{{ article.content }}</p>

    {% endfor %}

{% endblock %}
```

Les Templates DJANGO

- Inclure des sous-templates : include

```
# child.html
{% block content %}
{% for article in article_list %}
    {% include "app/content.html" with article=article %}
{% endfor %}
{% endblock %}

#content.html
<h2>{{ article.title }}</h2>
<p>{{ article.content }}</p>
```

Les Templates DJANGO

- **Charger des modules tiers : load**
- La balise `{% load %}` permet d'enrichir le moteur de template avec des balises et des filtres définis dans des modules supplémentaires déposés dans un dossier **templatetags**, situés à divers emplacements de django ou dans des emplacements créés par les développeurs
- Ces modules doivent contenir une variable `register` affecté avec une instance de `django.template.Library()`

```
# templatetags
from django import template
```

```
register = template.Library()
```

```
@register.filter()
```

```
def filter(value, [arg]) : # value est la variable à modifier, arg un paramètre de filtre
```

```
...
```

```
@register.simple_tag()
```

```
def tag([param]) : # cette fonction va produire du contenu
```

```
...
```

Les Templates DJANGO

- Modules tiers existants

`django / django / templatetags /`

`__init__.py`

`cache.py`

`i18n.py`

`l10n.py`

`static.py`

`tz.py`

Les modules d'extension de templates existants se trouvent disséminés dans le code de Django, toujours dans des dossiers « templatetags »

Ex : `django.django.templatetags`,
`django.contrib.humanize.templatetags`, ...

<https://github.com/django/django>

Les Templates DJANGO

- Les moteurs de templates
- l'entrée BACKEND permet d'insérer la classe de gestion des templates
- Ici DjangoTemplates ; on pourrait remplacer par Jinja2, ou un moteur personnalisé

```
TEMPLATES = [  
    {  
        'BACKEND': 'django.template.backends.django.DjangoTemplates',  
        'DIRS': [],  
        'APP_DIRS': True,  
        'OPTIONS': {  
            'context_processors': [  
                'django.template.context_processors.debug',  
                'django.template.context_processors.request',  
                'django.contrib.auth.context_processors.auth',  
                'django.contrib.messages.context_processors.messages',  
            ],  
        },  
    },  
]
```

Les Vues DJANGO

- La requête HTTP : attributs
 - Attributs principaux:
 - request.scheme : protocole http, https
 - request.body : corps sous forme d'octets
 - request.method : verbe HTTP (GET, POST, PUT, DELETE)
 - request.encoding : encodage (utf8)
 - request.content_type : type MIME de la requête
 - request.GET : dictionnaire des paramètres GET
 - request.POST : dictionnaire des paramètres POST
 - request.COOKIES : dictionnaire contenant les cookies
 - request.FILES : dictionnaire des fichiers uploadés
 - request.headers : entêtes de requête

Les Vues DJANGO

- La requête HTTP : méthodes
- Méthodes principales :

`HttpRequest.get_host()` : le serveur d'origine

`HttpRequest.get_port()` : le port réseau de la requête

`HttpRequest.get_full_path()` : le chemin de la requête

`HttpRequest.is_ajax()` : détecte une requête AJAX

`HttpRequest.is_secure()` : détecte une requête https

Les Vues DJANGO

- La réponse HTTP : attributs / méthodes
- Attributs principaux:
 - `response.content` : corps de la réponse, sous forme d'octets
 - `response.status_code` : code http (200 par défaut)
 - `response.charset` : Encodage de la réponse (UTF8 par défaut)
 - `Response.reason_phrase` : texte associé au code http (200 => OK, 404 => NOT FOUND)
- Constructeur de la réponse :

```
HttpResponse.__init__(content=b'', content_type=None, status=200, reason=None, charset=None)
```

Les Vues DJANGO

- La réponse HTTP : propriétés
- Le corps de la réponse peut être transmis par blocs comme dans un fichier

```
response = HttpResponse()
response.write("<p>premier paragraphe</p>")
response.write("<p>second paragraphe</p>")
return response
```

- Les entêtes de la réponse sont accessibles comme dans un dictionnaire

```
response = HttpResponse(my_data, content_type='application/vnd.ms-excel')
response['Content-Disposition'] = 'attachment; filename="foo.xls"' #téléchargement
response['Age'] = 120
del response['Age']
```

<https://developer.mozilla.org/fr/docs/Web/HTTP/Headers>

Les Vues DJANGO

- Les réponses en erreur personnalisées
- Django met à disposition des classes de réponses associées à un code http

```
from django.http import HttpResponseRedirect, HttpResponseRedirectNotFound

def my_view(request) :
    if foo:
        return HttpResponseRedirectNotFound('<h1> My Page not found</h1>') #404
    else:
        return HttpResponseRedirect('<h1>My Page was found</h1>') # 200
```

<https://docs.djangoproject.com/en/3.1/ref/request-response/#ref-httpresponse-subclasses>

Les Vues DJANGO

- Les réponses en erreurs génériques
- Django utilise des classes d'exceptions « HttpXXX » que django peut capturer pour renvoyer une réponse générique au client

```
from django.http import Http404  
from app.models import Article
```

```
def my_view(request) :  
    try:
```

```
        article = Article.objects.get(pk=1)  
    except Article.DoesNotExist: # exception capturée si la clé primaire n'existe pas  
        raise Http404("Article does not exist")
```

- Les vues génériques sont des fonctions issues du modules `django.vies.defaults`
- Dans l'exemple ci dessus, c'est la fonction `page_not_found()` qui est renvoyée avec un template par défaut « 404.html »

Les Vues DJANGO

- Les fonctions de raccourcis

- Django fournit un module `django.shortcuts` avec des fonctions de confort

- `render` applique un template avec son context, et renvoie une `HttpResponse`

```
render(request, template_name, context=None, content_type=None, status=None, using=None)
```

- `redirect` retourne une `HttpResponseRedirect` vers un modèle, une vue ou une URL en paramètre

```
redirect(to, *args, permanent=False, **kwargs)
```

- `get_object_or_404` retourne la fonction `get` du gestionnaire du modèle en paramètre, avec les paramètres optionnels, ou lève une exception `Http404`

```
get_object_or_404(model, *args, **kwargs)
```


Les URLconfs DJANGO

- **Algorithme**

- Quand une url est transmise à Django, la procédure suivante est exécutée :
- Django charge le module renseigné dans la variable de configuration `ROOT_URLCONF` du fichier `settings.py`
- Django observe le contenu de la variable `urlpatterns`, qui est une liste d'appels aux fonctions `path()` et `re_path()` (avec regex) du module `django.urls`
- Dès lors q'un des chemins décrit dans un de ses appels correspond à l'url demandée, cet appel est sélectionné
- La vue associée à cet appel est exécutée

Les URLconfs DJANGO

- Gestion des paramètres
- Il est possible de capturer des fragments de l'url, délimités par « / », comme paramètres passés à la vue : .../**<value>**/...
- Les valeurs de ces fragments peuvent être contrôlées par des **convertisseurs** (int, str, path...)
- Certains de ses convertisseurs sont propres à Django, comme le **slug** qui contrôle la regex [a-z0-9-]

```
from django.urls import path

from . import views

urlpatterns = [
    path('articles/', views.index),
    path('articles/<int:year>/', views.year_archive),
    path('articles/<int:year>/<int:month>/', views.month_archive),
    path('articles/<slug:slug>/', views.article_detail),
]
```

Les URLconfs DJANGO

- Gestion des expressions régulières
- On peut capturer les fragments d'url de manière plus précise avec les exession regulière et la fonction `re_path`
- On utilise la notation de groupe nommés `(?P<param>...)` pour transmettre l'expression capturée à la vue

```
from django.urls import path, re_path

from . import views

urlpatterns = [
    path('articles', views.index),
    re_path(r'^articles/(?P<year>[0-9]{4})/$', views.year_archive),
    re_path(r'^articles/(?P<year>[0-9]{4})/(?P<month>[0-9]{2})/$', views.month_archive),
    re_path(r'^articles/(?P<slug>[w-]+)/$', views.article_detail),
]
```

Les URLconfs DJANGO

- Inclusions d'urls externes

- On peut injecter dans la variable urlpatterns des variables issues d'autres fichiers d'urls, notamment ceux créés directement dans les applications, du moment que le même format est respecté
- On utilise pour cela la fonction `django.urls.include`
- Cela permet une gestion imbriquée, voire factorisée des fragments à plusieurs niveaux

dans le fichier pointé par ROOT_URLCONFIG

```
from django.urls import include, path
```

```
extra_patterns = [  
    path('report/', extern_views.report),  
]
```

```
urlpatterns = [  
    path('admin/', admin.site.urls), # application admin créée par défaut avec le projet  
    path('app/', include('app.urls')), # correspond au fichier d'url dans /app.urls.py  
    path('extern<int:id>/', include(extra_pattern)) # passage des paramètres à la conf incluse  
]
```

Les URLconfs DJANGO

- Paramètres supplémentaires

- On peut ajouter dans les fonctions path et re_path des paramètres supplémentaires sous la forme de dictionnaire
- Ces paramètres seront inclus comme paramètres des vues appelées

```
# dans urls.py
from django.urls import path

from . import views

urlpatterns = [
    path('articles/', views.index, {"foo" : "bar"}),
]

# dans views.py
def index(foo) :
    pass
```


Les URLconfs DJANGO

- Tags d'URL

- Fonctionnalité nommée « **URL Reversing** » en anglais, permet d'associer un nom ou tag à une url, via le paramètre nommé **name** pour pouvoir exploiter simplement l'url complète :
- Dans les template avec la balise `{% url %}`
- Dans le code python avec la fonction `django.urls.reverse`
- Dans les modèles Django avec la fonction `Model.get_absolute_url`

```
# dans urls.py
```

```
...
```

```
path('articles/<int:year>', views.archive_year, name="year-articles"),
```

```
# dans article.html
```

```
...
```

```
{% for y in year_list %}
```

```
    <li><a href="{% url 'year-articles' y %}">{{ yearvar }} Archive</a></li>
```

```
# dans views.py
```

```
...
```

```
return HttpResponseRedirect(reverse('year-articles', args=[2020]))
```


Les URLconfs DJANGO

- **Espaces de Noms d'URL**

- L'espace de noms permet de taguer des listes entières de configuration d'url, tag qui sera complété par le tag de l'url individuelle
- On peut créer un espace de nom dès la création des configuration ou à l'inclusion
- Permet de différencier de noms d'url identiques

dans urls.py

```
extra_patterns = ([  
    path('report/', extern_views.report, name='report'),  
], 'extern')
```

```
urlpatterns = [  
    path('app/', include('app.urls', namespace='app'))  
    path('extern<int:id>/', include(extra_pattern)) # permet d'accéder à extern:report  
]
```

#dans article.html

...

```
<a href="{% url 'extern:report' %}">...
```

Les Vues Génériques

- Vues de base

- On peut écrire des vues sous forme de classes, héritant de classes de base assurant des cas d'utilisation standards
- La classe la plus simple est la classe View, qui possède des méthodes surchargeables, via `super()` :
- `setup(request, *args, **kwargs)` : initialisation des variables et objets de la vue
- `dispatch(request, *args, **kwargs)` : examine la méthode http demandée (GET, POST, PUT...) et tente de lancer une méthode de classe du même nom
- `http_method_not_allowed(request, *args, **kwargs)` : si la méthode n'est pas codée
- `options(request, *args, **kwargs)` : traite les réponses spécifiques aux requêtes OPTIONS
- Méthodes acceptées : ['get', 'post', 'put', 'patch', 'delete', 'head', 'options', 'trace']

```
from django.http import HttpResponseRedirect
from django.views import View
```

```
class MyView(View):
```

```
    def get(self, request, *args, **kwargs):
        return HttpResponseRedirect('Hello, World!')
```

Les Vues Génériques

- Appel d'une vue comme classe
- On peut associer la classe dans les configurations d'urls en utilisant la méthode `asView`

```
from django.urls import path

from myapp.views import MyView

urlpatterns = [
    path('mine/', MyView.as_view(), name='my-view'),
]
```

Les Vues Génériques

- **Vue de base avec Template**
- Même fonctionnement que View mais pour afficher un template avec son contenu (GET)

```
from django.views.generic.base import TemplateView
from articles.models import Article
```

```
class HomePageView(TemplateView):
```

```
    template_name = "home.html"
```

```
    def get_context_data(self, **kwargs):
        context = super().get_context_data(**kwargs)
        context['latest_articles'] = Article.objects.all()[:5]
        return context
```

Les Vues Génériques

- Vue de base avec Redirection

- Même fonctionnement que View avec un mécanisme de redirection
- Les attributs `url` ou `pattern_name` permettent de signifier l'url de redirection
- l'attribut `permanent` pour le type de redirection
- l'attribut `querystring` pour conserver la querystring dans la redirection
- La méthode `get_redirect_url` surchargée permet de faire des traitements avant redirection

```
from django.shortcuts import get_object_or_404
from django.views.generic.base import RedirectView
from articles.models import Article
```

```
class ArticleCounterRedirectView(RedirectView):
```

```
    permanent = False
    query_string = True
    pattern_name = 'article-detail'
```

```
    def get_redirect_url(self, *args, **kwargs):
        article = get_object_or_404(Article, pk=kwargs['pk'])
        article.update_counter()
        return super().get_redirect_url(*args, **kwargs)
```


Les Vues Génériques

- **Vue d'affichage : DetailView**

- Sert à afficher un enregistrement d'un model de donnée => vue fiche
- Mêlé des propriétés de View, TemplateView, en ajoutant en paramètre un Model
- La vue possède un attribut **self.object** qui décrit l'objet décrit par la vue
- l'indice de l'enregistrement est injecté par l'urlconf ex : « **article/<int:pk>** »

```
from django.utils import timezone
from django.views.generic.detail import DetailView

from articles.models import Article

class ArticleDetailView(DetailView):

    model = Article
    template_name = 'app/fiche.html'

    def get_context_data(self, **kwargs):
        context = super().get_context_data(**kwargs)
        context['now'] = timezone.now()
        return context
```

```
# app/fiche.html

<h1>{{ object.headline }}</h1>

<p>{{ object.content }}</p>

<p>Reporter: {{ object.reporter }}</p>

<p>Published: {{ object.pub_date|date
}}</p>

<p>Date: {{ now|date }}</p>
```


Les Vues Génériques

- **Vue d'affichage : ListView**

- Sert à afficher une liste d'enregistrements d'un model de donnée => vue liste
- Mêlé des propriétés de View, TemplateView, en ajoutant en paramètre un Model
- La vue possède un attribut `self.object_list` qui décrit la liste d'objets décrits par la vue
- La vue fournit un mécanisme de pagination automatique

```
from django.utils import timezone
from django.views.generic.list import ListView
```

```
from articles.models import Article
```

```
class ArticleListView(ListView):
```

```
    model = Article
```

```
    paginate_by = 100 # optionnel
```

```
# app/article_list.html
```

```
<h1>Articles</h1>
```

```
<ul>
```

```
{% for article in object_list %}
```

```
    <li>{{ article.pub_date|date }} -
```

```
    {{ article.headline }}</li>
```

```
{% empty %}
```

```
    <li>No articles yet.</li>
```

```
{% endfor %}
```

```
</ul>
```

Les Fichiers statiques

- configuration

- Les fichiers statiques sont gérés par les variables de configurations suivantes :
- **STATIC_ROOT** : dossier central des fichiers statiques, utilisé pour la collecte et le déploiement
- **STATIC_URL** : le chemin de l'url apparente dans les liens des fichiers statiques
- **STATICFILES_DIR** : dossiers supplémentaires dans lesquelles django va rechercher des chemins de fichiers statiques
- **STATICFILES_STORAGE** : moteur de stockage de fichiers utilisé lors de la collecte. Par défaut `django.contrib.staticfiles.storage.StaticFilesStorage`
- **STATICFILES_FINDERS** : classes de recherche des fichiers statiques. Par défaut `'django.contrib.staticfiles.finders.FileSystemFinder'` et `'django.contrib.staticfiles.finders.AppDirectoriesFinder'`
- une classe recherche dans les **STATICFILES_DIR**, l'autre dans les dossier « static » des applications

Les Fichiers statiques

- Dans les templates

```
STATIC_URL = '/static/'
STATICFILES_DIRS = (
    os.path.join(BASE_DIR, 'static'),
)
STATIC_ROOT = os.path.join(BASE_DIR, 'staticfiles')
```

- Une fois la configuration réglée, on peut utiliser les tags du module static dans les templates pour faciliter l'écriture des liens de fichiers statiques

```
{% load static %}
...
<link rel="stylesheet" href="{% static 'css/custom.css' %}">
...

...
<script src="{% static 'js/custom.js' %}"></script>
```

Données initiales

- « fixtures »
 - On appelle données initiales ou fixtures des fichiers contenant des objets correspondants aux modèles de données Django
 - Ces fixtures doivent être placées par défaut dans un dossier « fixture » dans chaque application
 - On peut ajouter une liste de dossier de fixtures avec la variable de configuration `FIXTURE_DIRS`
 - Si la base est déjà peuplée de données, on peut générer des fixtures en utilisant `manage.py dumpdata`
 - On peut créer les fixtures à la main dans plusieurs format comme XML, JSON, YAML...
 - On charge les fixtures avec `manage.py loaddata <nom du fichier>`

Données initiales

- exemple

```
#dans fixtures/person.json
[
  {
    "model": "myapp.person",
    "pk": 1,
    "fields": {
      "first_name": "John",
      "last_name": "Lennon"
    }
  },
  {
    "model": "myapp.person",
    "pk": 2,
    "fields": {
      "first_name": "Paul",
      "last_name": "McCartney"
    }
  }
]
```


Les Fichiers statiques

- La collecte

- La collecte consiste à parcourir l'ensemble des dossiers static du projet pour tout rassembler dans le dossier point par **STATIC_ROOT**
- Il est important de créer des sous répertoires « espaces de noms » pour éviter les doublons de fichiers statiques qui ne sont pas gérés et seront donc systématiquement écrasés par le moteur de stockage
- Une fois la collecte exécutée, l'ensemble des fichiers statiques peut être transféré vers un dossier géré par un serveur web dédié, ou un service CDN distant => **déploiement**
- On peut automatiser cette dernière opération en modifiant le moteur de stockage dans **STATICFILES_STORAGE**